GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

Bringing Humanoids to LeRobot



Bringing Humanoids to LeRobot

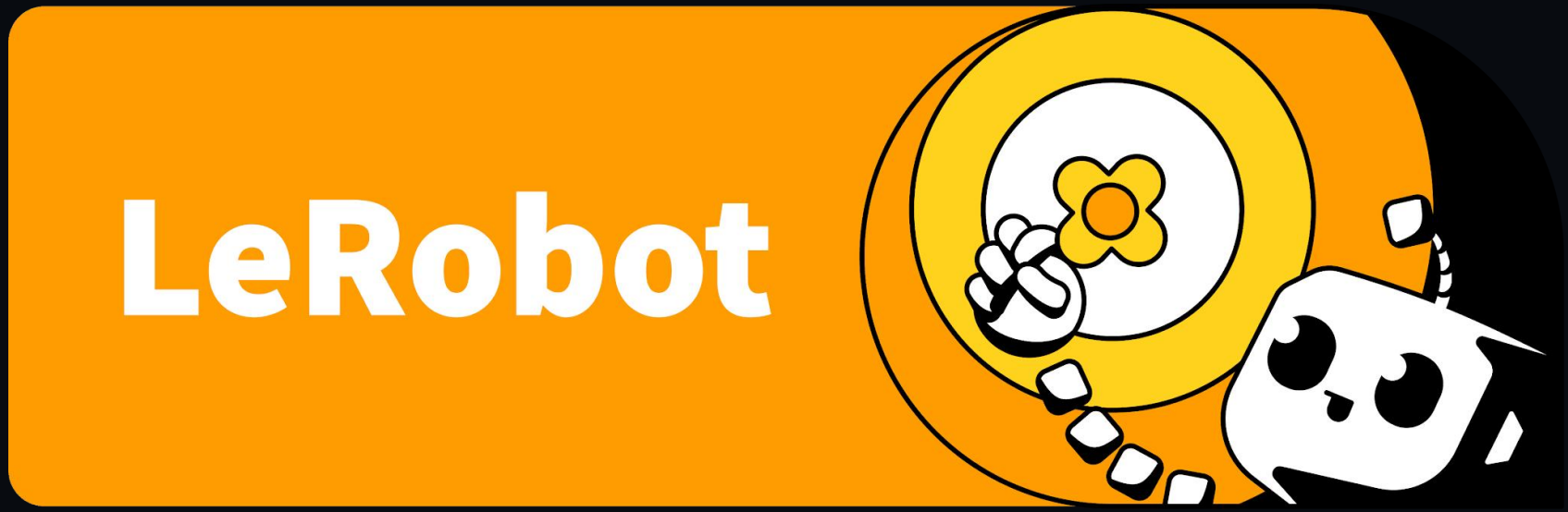




Martino Russi

Embodied robotics engineer
Hugging Face

LeRobot



An open-source library for robot learning

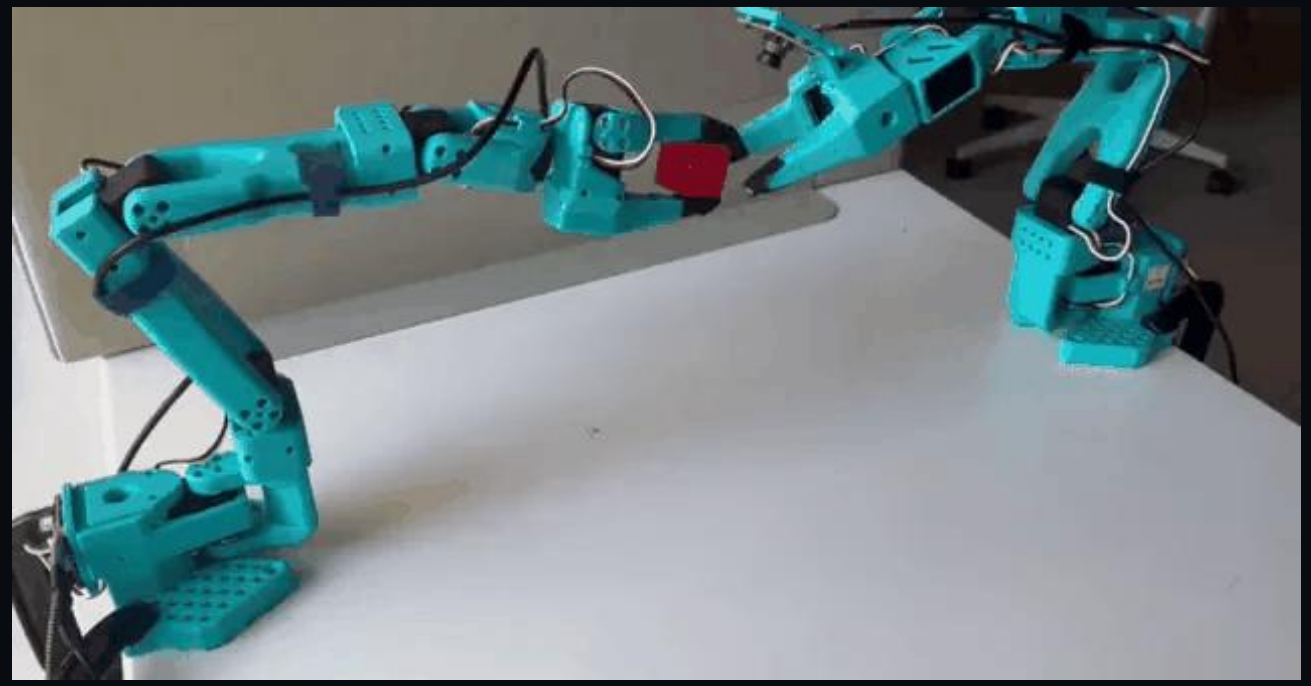
LeRobot combines hardware, datasets and models together through a shared pipeline.

I maintain LeRobot and focus on humanoid SW/HW.

LeRobot

Connecting open-source robotics into a shared workflow

01 Robot interfaces | 02 Datasets | 03 Policies



Robots and teleoperators

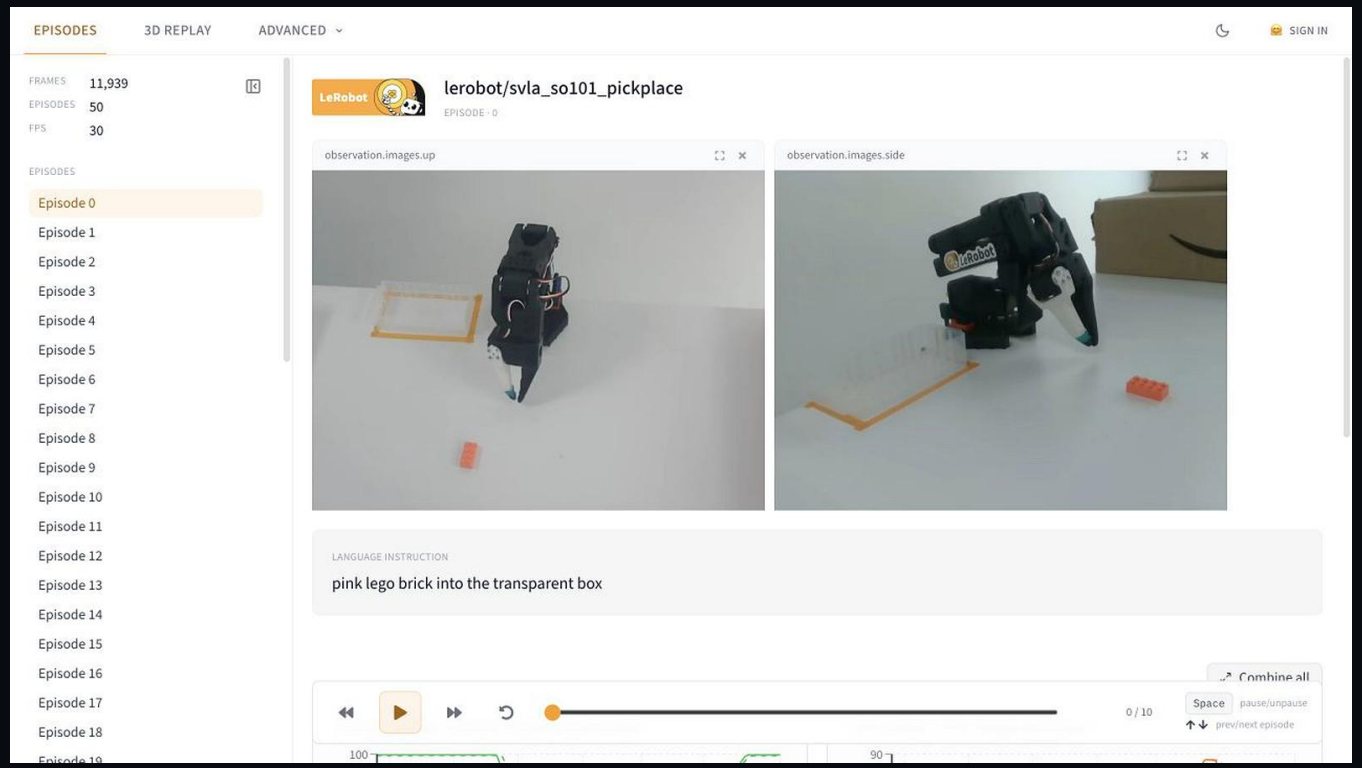
Koch, SO-101, LeKiwi, OpenArm, ReBot, Unitree G1

Shared interfaces for recording, training and deployment.

LeRobot

Connecting open-source robotics into a shared workflow

01 Robot interfaces | 02 Datasets | 03 Policies



Synchronized multimodal data

RGB and metric depth cameras
States, actions and language
Tactile, Audio..

Compressed video + Parquet
Smaller files; synchronized episodes
Stream directly from the Hub

LeRobot

Connecting open-source robotics into a shared workflow

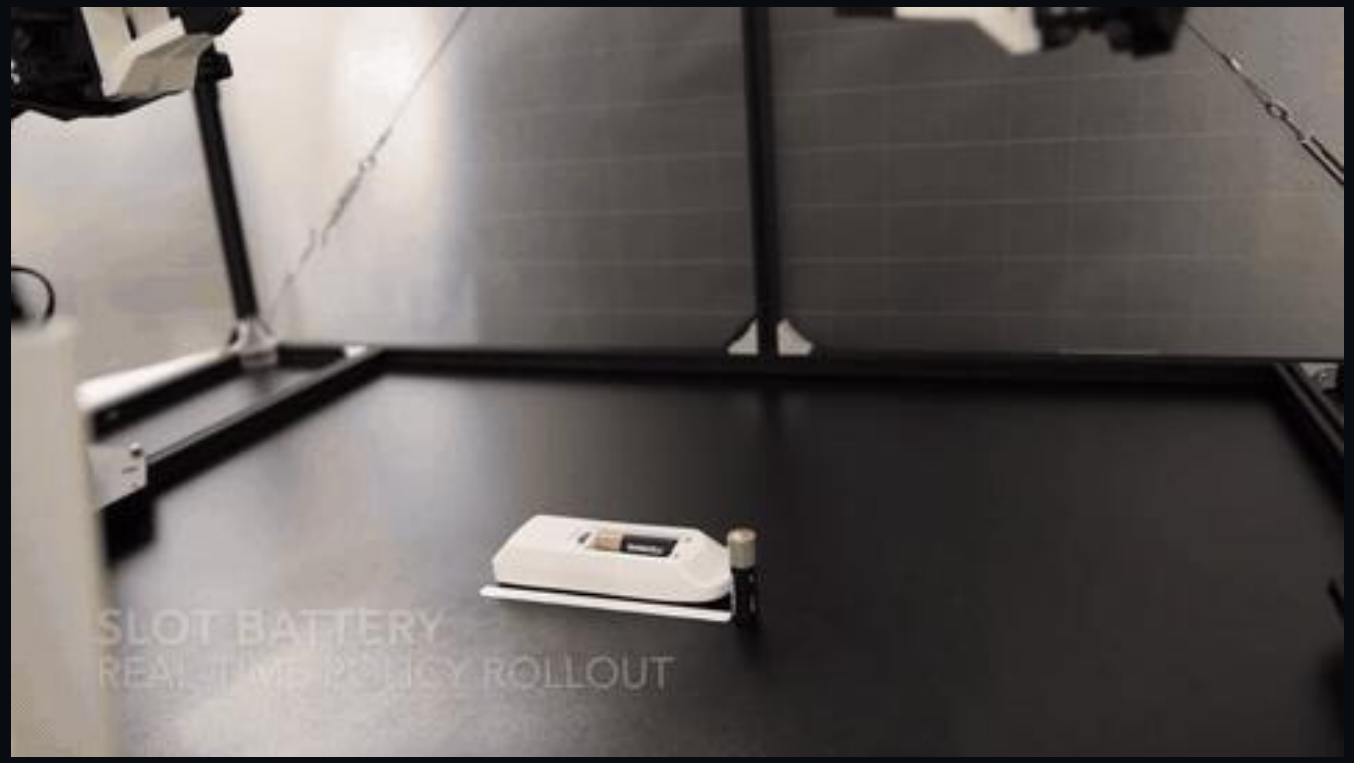
01 Robot interfaces 02 Datasets 03 Policies

Imitation learning

ACT

Action Chunking Transformer

Learns from teleoperated demos.
Predicts a sequence
of joint targets.



ALOHA battery insertion · ACT · 50 demonstrations

LeRobot

Connecting open-source robotics into a shared workflow

01 Robot interfaces 02 Datasets 03 Policies

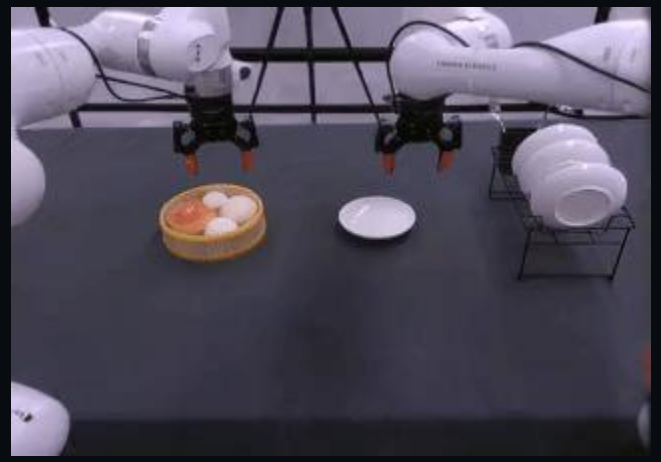
Imitation learning

ACT

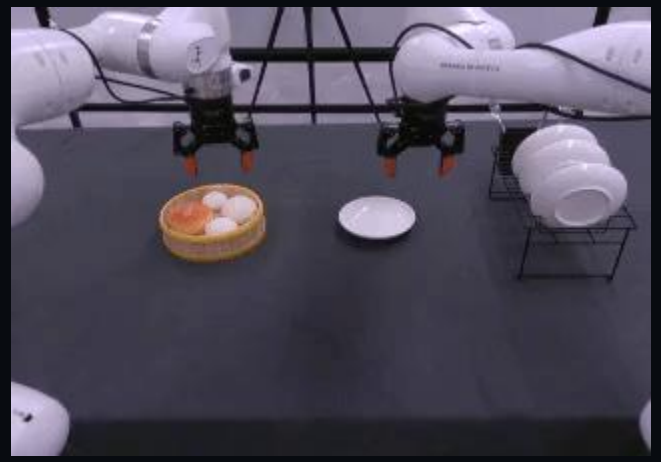
World / action models

FastWAM · LingBot-VA
VLA-JEPA

LingBot-VA prediction



Robot observation



LeRobot

Connecting open-source robotics into a shared workflow

01 Robot interfaces

02 Datasets

03 Policies

Imitation learning

ACT

World / action models

FastWAM · LingBot-VA
VLA-JEPA

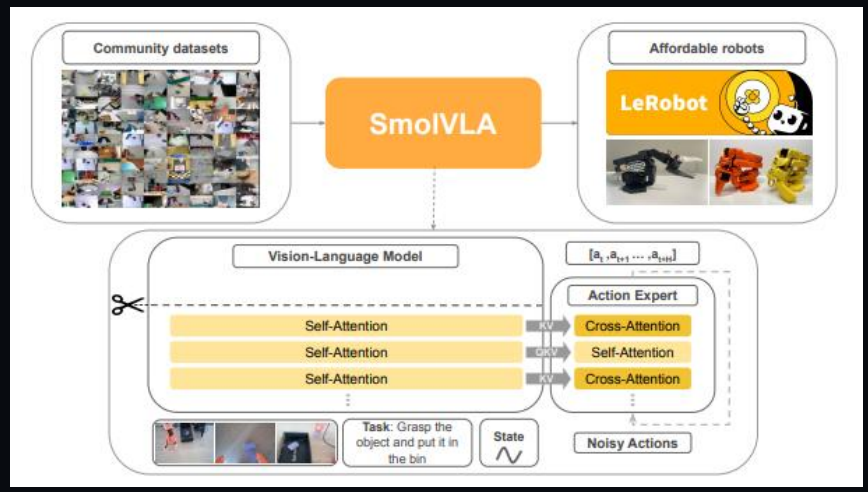
Vision-Language-Action

SmolVLA · π0 · π0.5

VLA example: π0 folding clothing

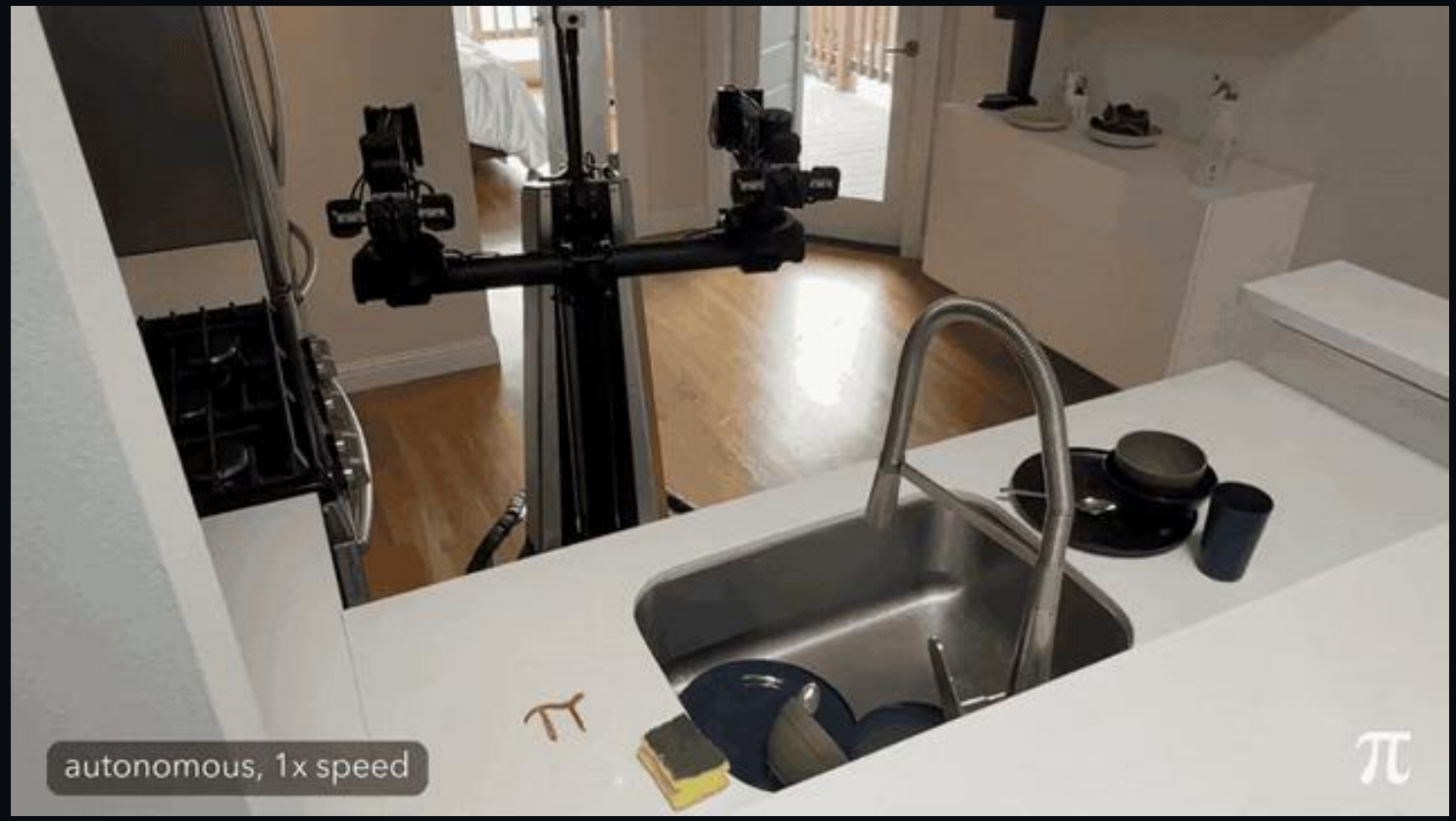


SmolVLA architecture



π0.5

Physical Intelligence's π0.5 on a mobile manipulator



Physical Intelligence · original speed annotations retained

Clean up the spill

Images, language and state condition the action expert. It predicts a chunk of motion.

Observe again and generate the next action chunk.

How π0.5 works

01 Observations

02 VLM context

03 Action expert



clean up the spill



Observation prefix

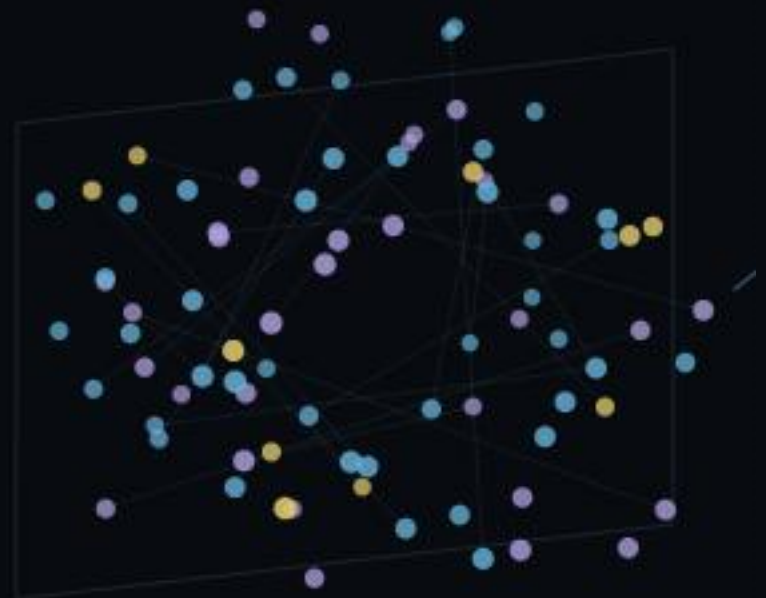
Model input consists of visual tokens, language description and robot state.

The model conditions on these inputs together.

Illustrative projections, not measured model activations

How π0.5 works

01 Observations | 02 VLM context | 03 Action expert



PaliGemma backbone

Image, language and state tokens form a shared context.

Compute prefix keys and values once. Reuse this cache while the action expert generates a chunk.

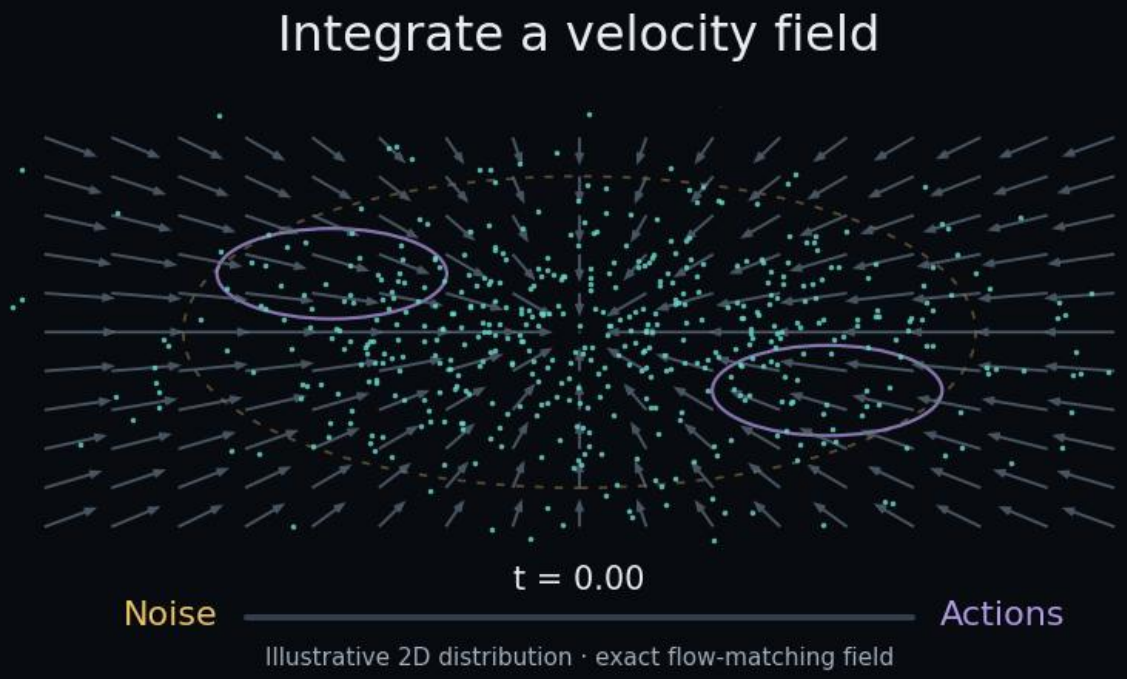
Illustrative projections, not measured model activations

How π0.5 works

01 Observations

02 VLM context

03 Action expert



Flow matching in the action head

$$dx/dt = v_\theta(x, t \mid o)$$
$$x(0) \sim N(0, I)$$

$$x(1) = x(0) + \int_0^1 v_\theta \, dt$$

Integrate the learned velocity
to generate the whole action chunk.

x = action chunk · o = observations and instruction · t = flow time

Why action chunks?

3 camera views

Batch compatible images in one pass.
Build the prefix once; cache its context.

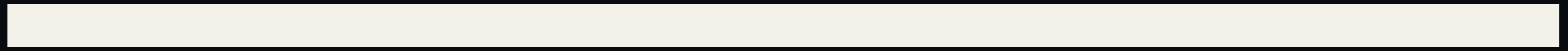
10 flow steps

Reuse the context at every step.
Generate all 50 actions together.

≈272 ms to predict an action chunk



50 actions = 1 second of motion at 50 Hz



≈13.6 controller ticks per prediction: overlap inference and execution

Reported G1 checkpoint timing · compute call, not end-to-end latency

Real-time chunking

Start the next prediction before the queue empties

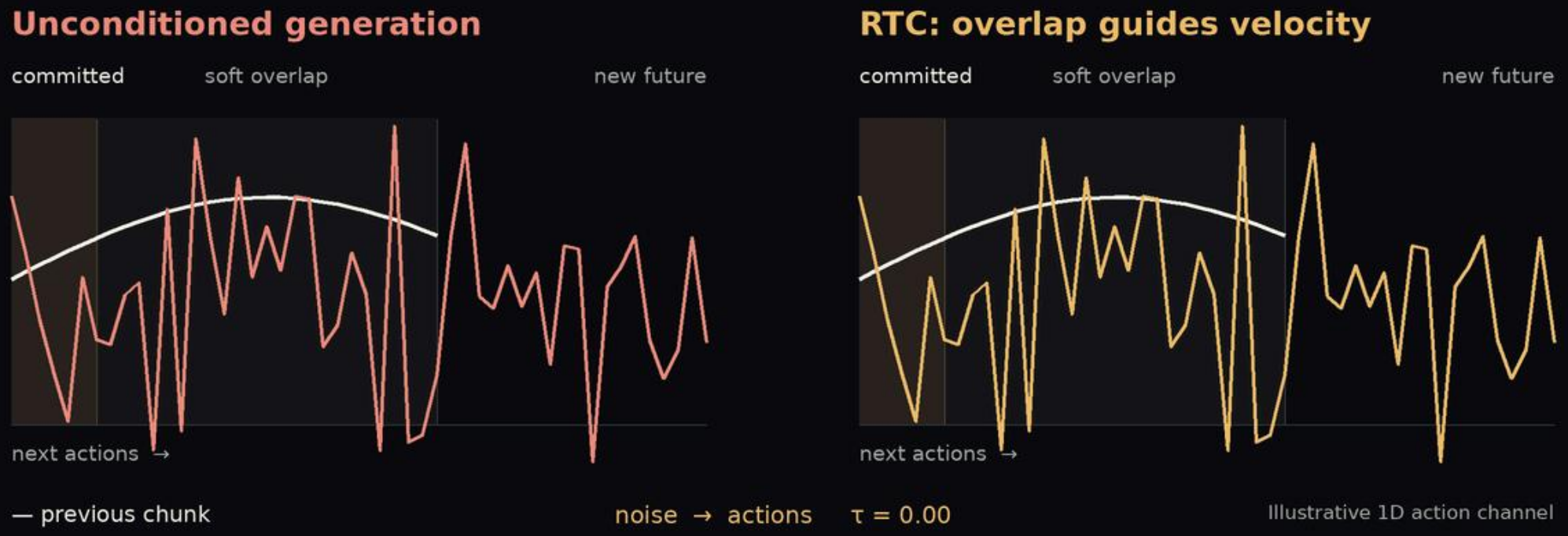


Guided RTC: inference-time guidance Trained RTC: learned prefix conditioning

RTC: conditioning the overlap

$$v_{RTC} = v_\theta - \lambda(t)\ \nabla_x L_{overlap}$$

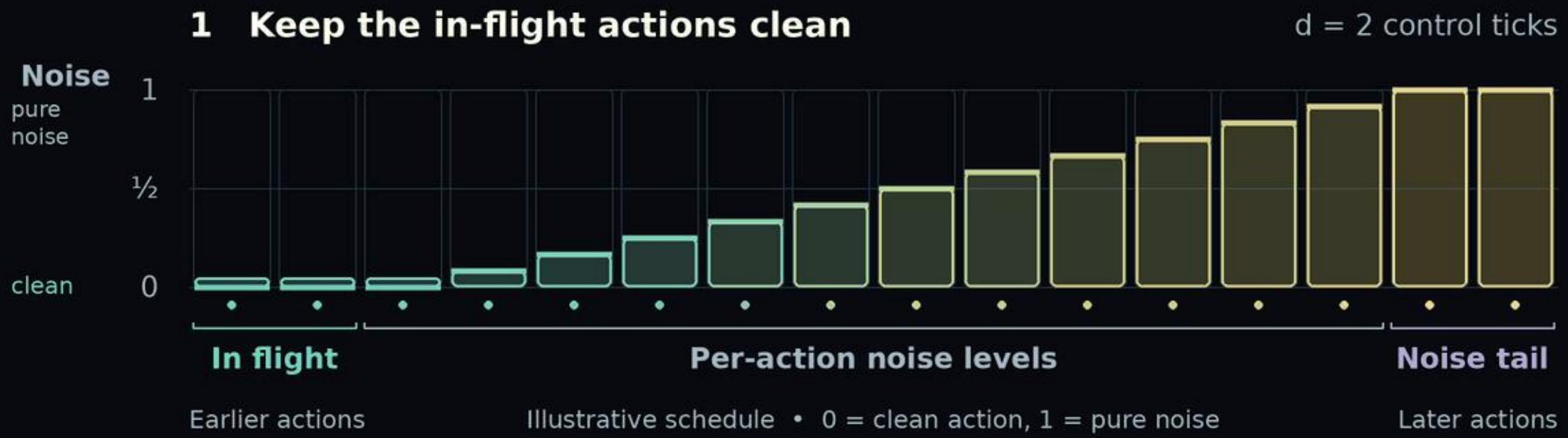
The previous chunk guides the velocity in the overlap.
Integrate that corrected velocity to continue the motion.



πR²: reactive flow policies

Fast body feedback

Slower vision-language context



Fresh body state each update; vision-language context refreshes asynchronously

Requires a policy fine-tuned for this noise schedule

VLA inference and robot control

Constraint	Execution mechanism
Vision and prefix cost	Batching images, cache context, and generate action chunks in the background.
The world keeps changing	RTC and πR² increase reactivity

Inference optimizations improve execution; they do not remove all reaction delay.

VLA inference and robot control

Constraint	Execution mechanism
Vision and prefix cost	Batching images, cache context, and generate action chunks in the background.
The world keeps changing	RTC and πR² increase reactivity
Balance and contacts	???

Inference optimizations improve execution; they do not remove all reaction delay.

Unitree G1: capabilities and integration



What it unlocks	What it requires
Mobile, bimanual work	Balance while the arms and body move
29-joint whole-body motion	Fast joint and IMU feedback through a controller
Human-motion teleoperation	Retargeting and synchronized observations and actions

LeRobot supports: observation interfaces, upper-body gravity compensation, arm IK and VR teleoperation, shared across hardware and simulation.

Controllers

Desired motion

VLA action, VR reference
or velocity command

Measured body

Joint positions and velocities
IMU and previous actions

Learned controller

Joint targets with continuous feedback

$$\tau = Kp\,(qtarget - q) - Kd\,\dot{q}$$

Low-level PD tracking · the next step reads the body again

Whole-body controllers in LeRobot

SONIC

Latent-conditioned whole-body tracking

GR00T WholeBodyControl

Upper-body IK + lower-body RL

Holosoma

RL locomotion and motion tracking



The policy’s action representation must match its deployment controller.

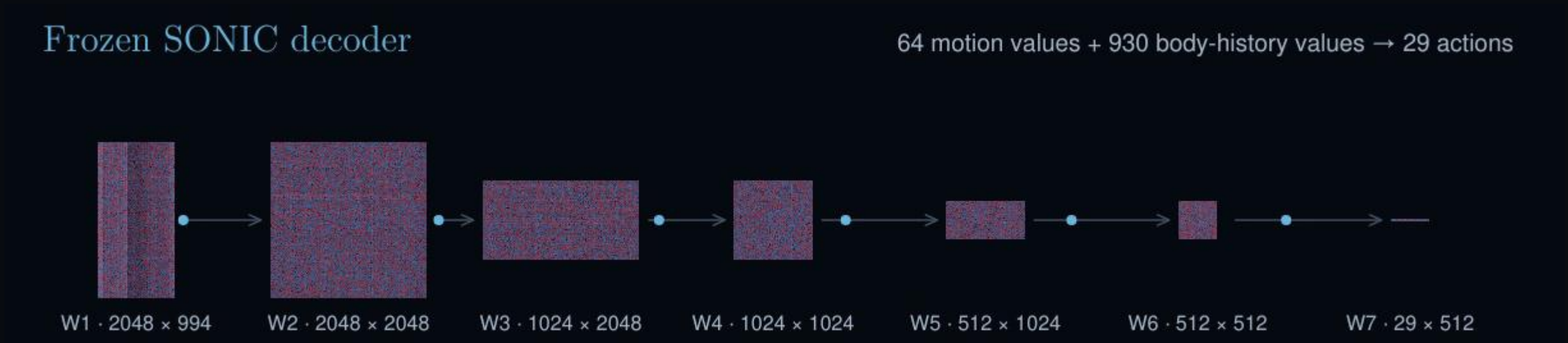
Where SONIC's 64 motion values come from

BONES-SEED motion, retargeted to G1, trains the motion representation.
Finite scalar quantization rounds the encoder output to discrete levels.



Released SONIC: 2 tokens × 32 dimensions = 64 values · 32 quantization levels per scalar

SONIC: motion tokens and body feedback



Motion input

64-D motion representation

Two 32-D tokens are flattened into the controller’s motion input.

Body feedback

Proprioceptive history

The decoder produces joint targets at 50 Hz.

How SONIC controls whole-body motion

Stage	Mechanism
Learn motion tracking	Track BONES-SEED human motion, retargeted to G1, with PPO in simulation.
Encode the reference	Motion clips, VR or human skeletons encode into two quantized 32-D tokens.
Decode with feedback	Motion token + recent body state produce 29 joint commands at 50 Hz.

A task policy chooses motion; SONIC keeps tracking it with fresh body feedback

OpenHLM

OpenHLM: VLA → controller

Desired whole-body poses

The controller tracks the requested motion

Our LeRobot adaptation

64-D SONIC tokens + 2 grippers

Predict the controller's motion representation





The physical G1 setup



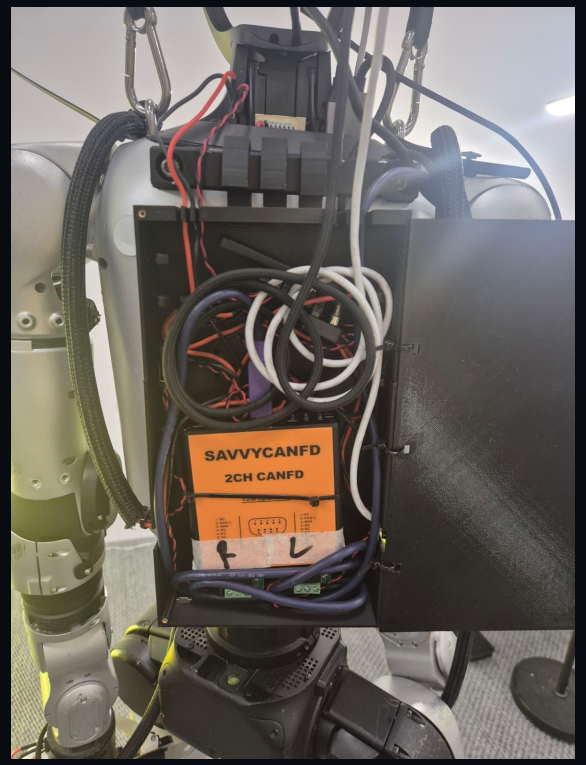
OpenArm grippers

Printed wrist adapters



Printed components

CAD for the G1 integration



Backpack electronics

CAN-FD and gripper wiring

Collecting the G1 demonstrations

3	31	66
camera streams	state values	action values

State

29 joint positions + 2 grippers

Action

64 SONIC latent values
+ 2 gripper commands

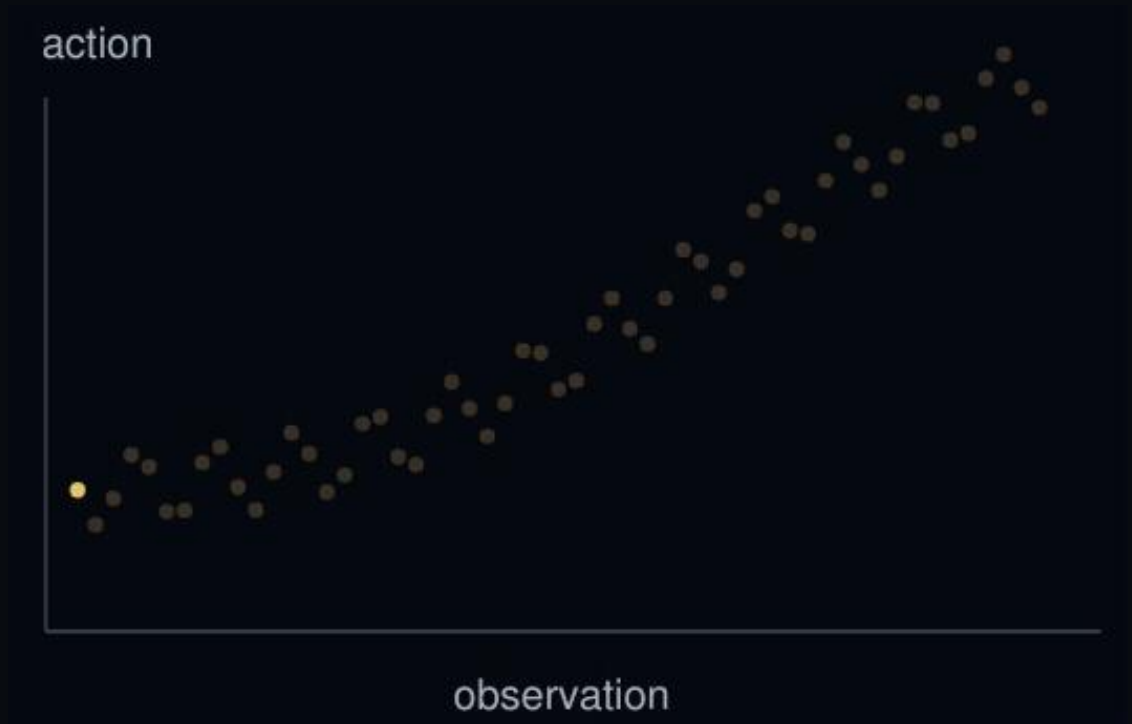
50 Hz synchronized episodes in LeRobotDataset

Learning the can-transfer task

01 Collect demonstrations

02 Fine-tune π0.5

03 Execute the policy



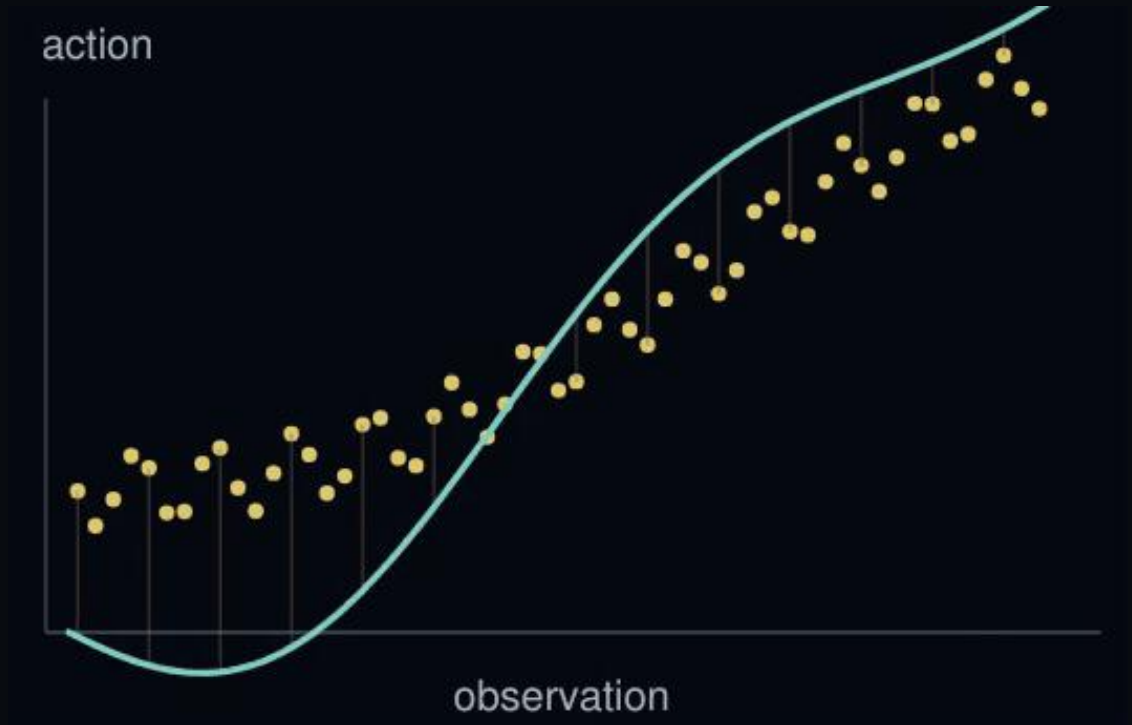
Observation–action pairs



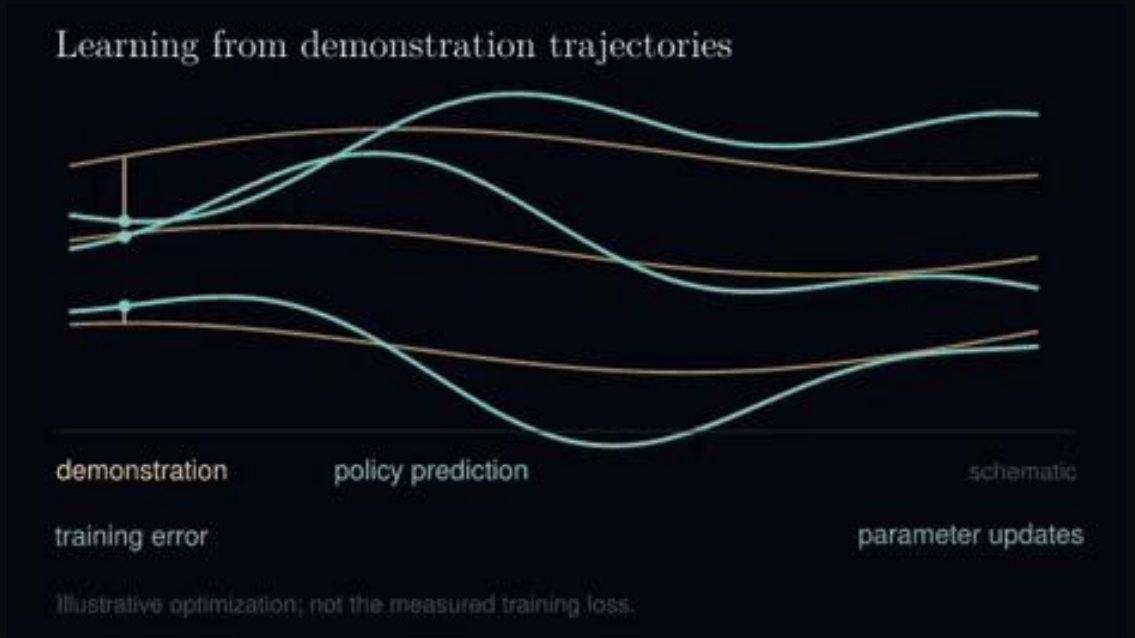
105 demonstrations, 70.8 minutes

Learning the can-transfer task

01 Collect demonstrations | 02 Fine-tune π0.5 | 03 Execute the policy



Fit the policy to demonstrations



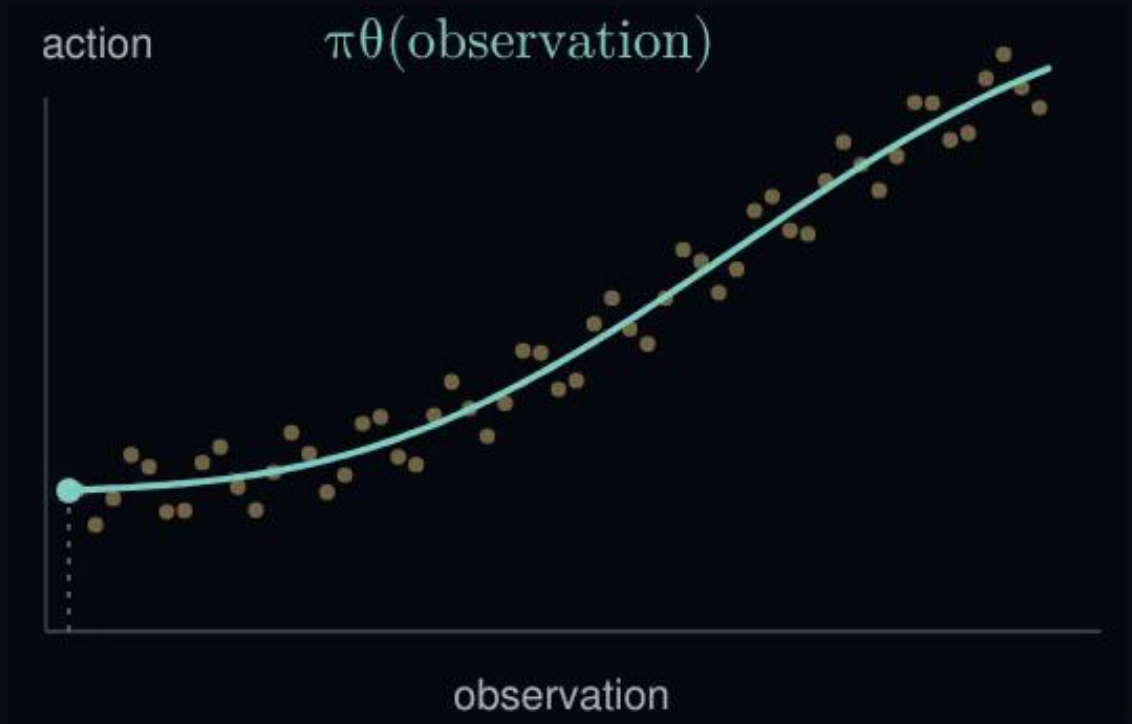
12,000 steps on 4 × H100

Learning the can-transfer task

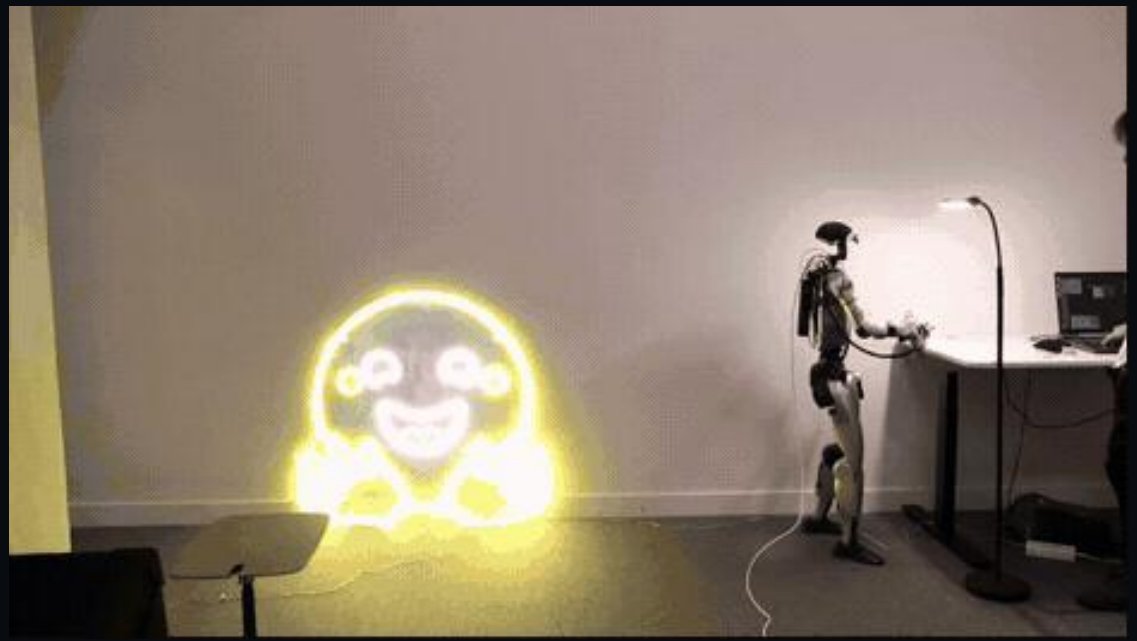
01 Collect demonstrations

02 Fine-tune π0.5

03 Execute the policy



Predict actions from observations



Learned can-transfer behavior

The VLA and controller run asynchronously

GPU policy server

≈272 ms

This G1 setup uses a second machine

Onboard SONIC controller

20 ms

One 50 Hz controller tick



Execute buffered actions while the next prediction is computed

50 predicted steps = 1 s of motion · computation and network delays still matter

Async inference: managing the action queue

While the robot moves	Background work
Request before empty	When the buffer runs low, send fresh images, state and the playback cursor.
Predict without blocking	The GPU builds the next chunk while SONIC keeps tracking at 50 Hz.
Align the returned chunk	Discard committed steps; update future actions. RTC conditions on the overlap.

Async overlaps compute and motion; RTC makes the new chunk fit the ongoing motion

VLA latency and fast visual reactions

Optimized VLA inference

Batch camera images; cache the prefix.
Overlap inference with execution.

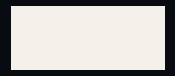
What still remains

New images affect actions only after inference and the chunk handoff.

≈272 ms: one π0.5 prediction



20 ms: one 50 Hz control period



What if we ran everything onboard?

Example compute timing, not end-to-end reaction latency or a universal VLA limit

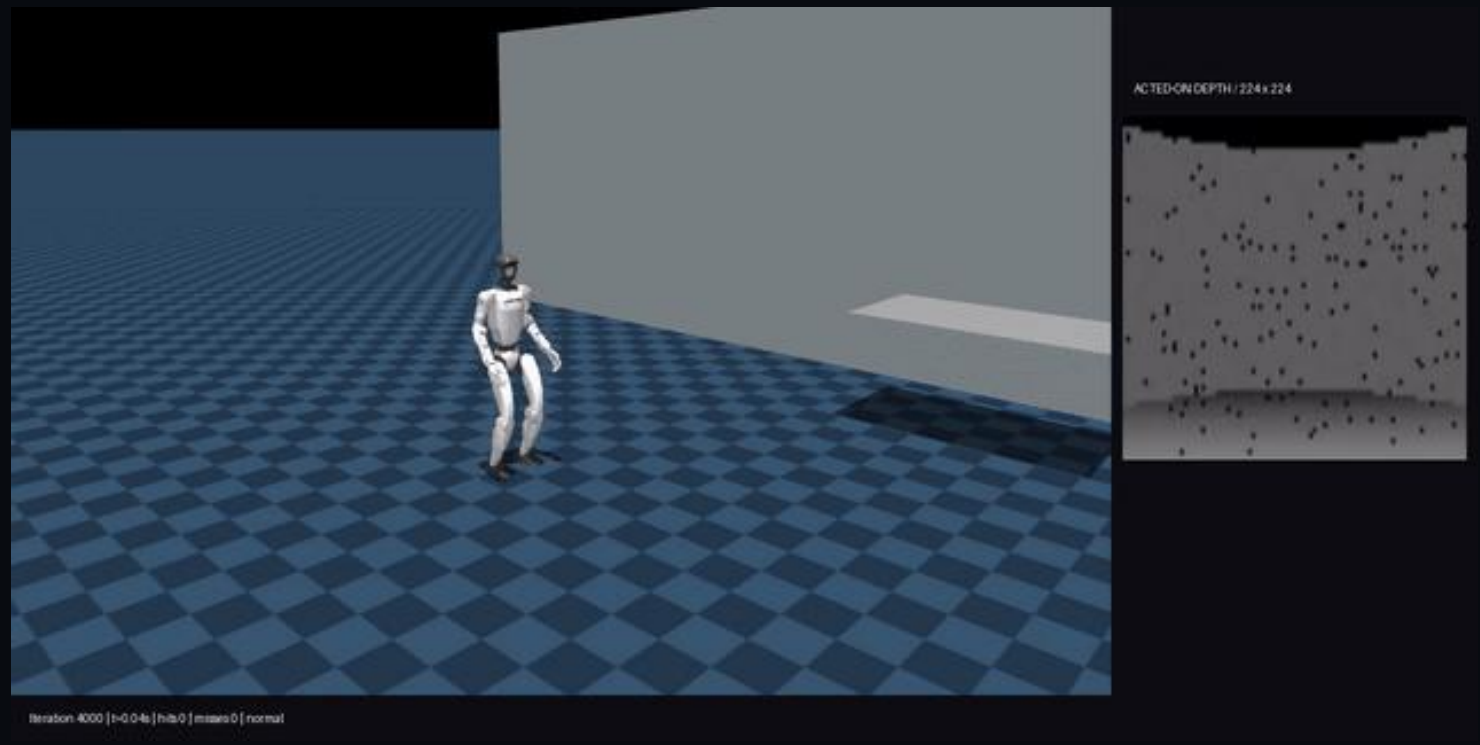
Building on ViBe and ORCS

Source	Contribution
ViBe · original method	Visual feedback and low-rank adaptation of pretrained whole-body trackers.
ORCS · open toolkit	Optimize, Retarget, Control Suite: training, retargeting and SONIC adaptation.
This Depth Dodge project	Learn a temporal percept from depth, adapt SONIC, and deploy through LeRobot.

ViBe: Visual Behavior Adaptation for Perceptive Humanoid Whole-Body Control

Training G1 in sim

Avoid a moving ball while staying balanced



Recorded simulation · depth input shown in the inset

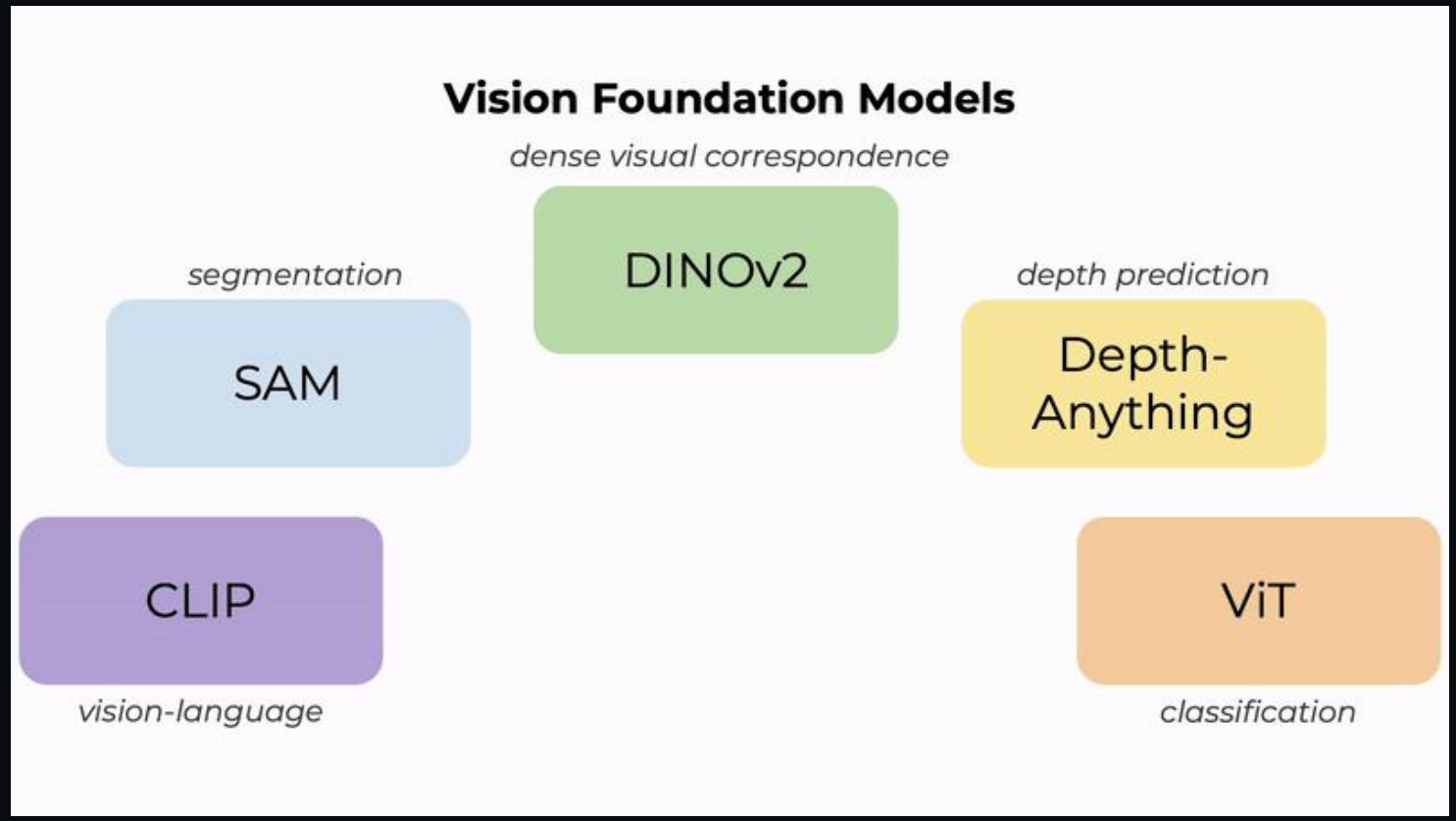
A different control task

We feed depth information directly to the controller

How?

Theia-Tiny: visual features for control

Distill several vision models into one compact representation



Theia overview animation · The AI Institute

Frozen DeiT-Tiny

≈5.5M frozen parameters
224 × 224 depth image
197 × 192 visual tokens

Freeze the visual encoder.
Train the temporal adapter
and SONIC LoRAs.

From depth history to a 128-D percept

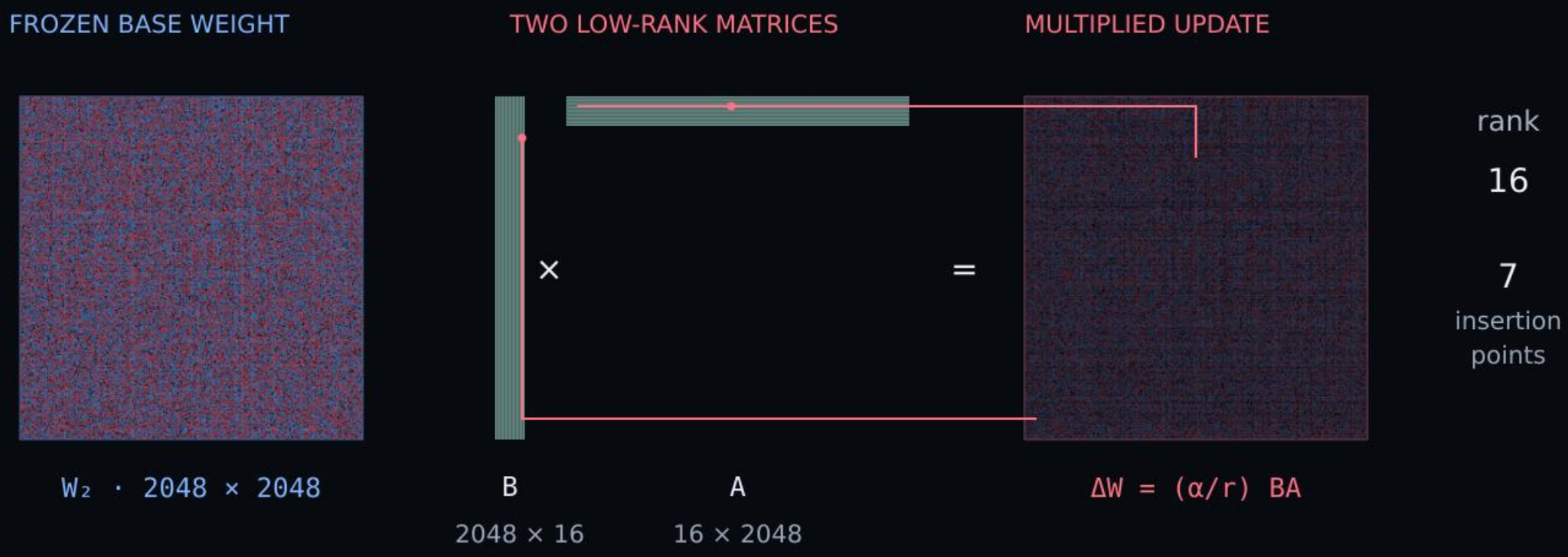
Learned pooling selects spatial features; temporal mixing captures the approach.



624,769 learned parameters · ball position and velocity supervise the percept during training

LoRA adaptation of the SONIC decoder

The first LoRA injects vision; the remaining six adapt decoder hidden layers.

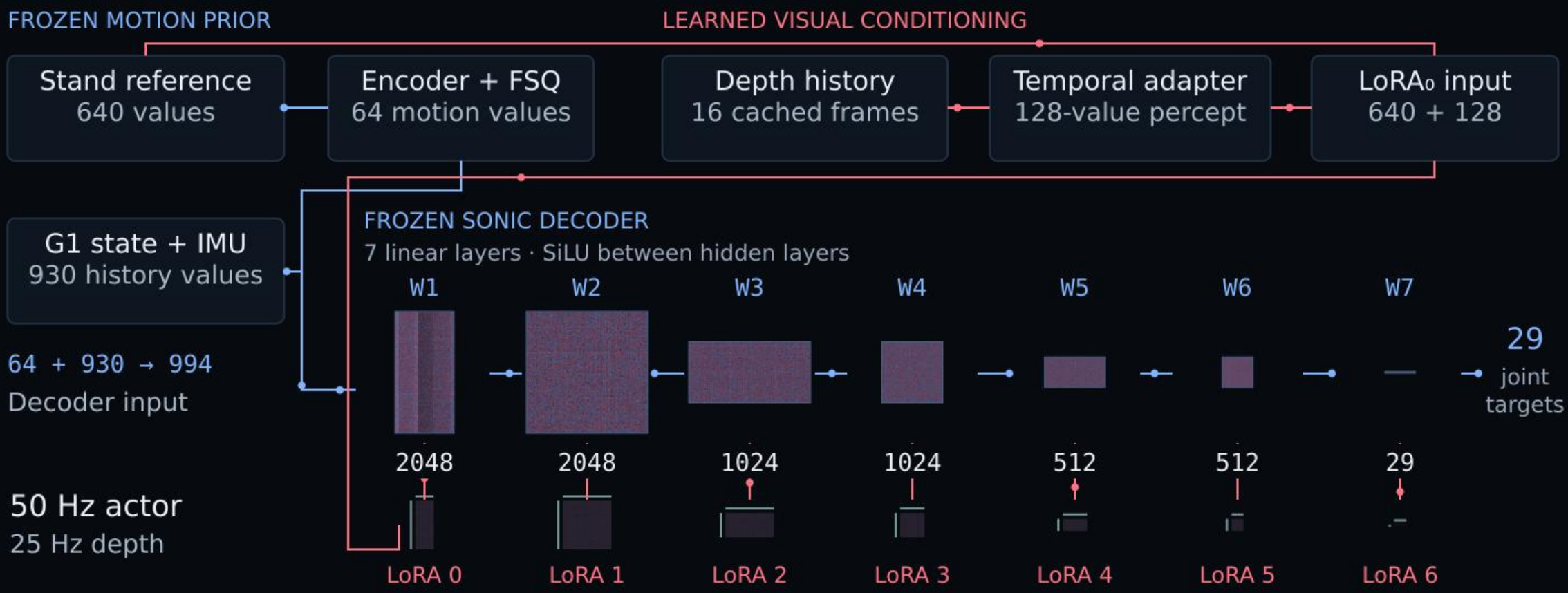


Actual W / ΔW · A/B rank dimension enlarged for visibility; individual values unavailable

LoRA$_0$: reference (640) + visual percept (128) → 768 conditioning values

Depth Dodge policy architecture

A learned visual percept adapts a frozen whole-body controller



Train the temporal adapter and seven rank-16 LoRAs; keep Theia and SONIC frozen.

Depth perception for dodging

Recorded depth input

Temporal-adapter pooling



16 history samples · oldest

newest



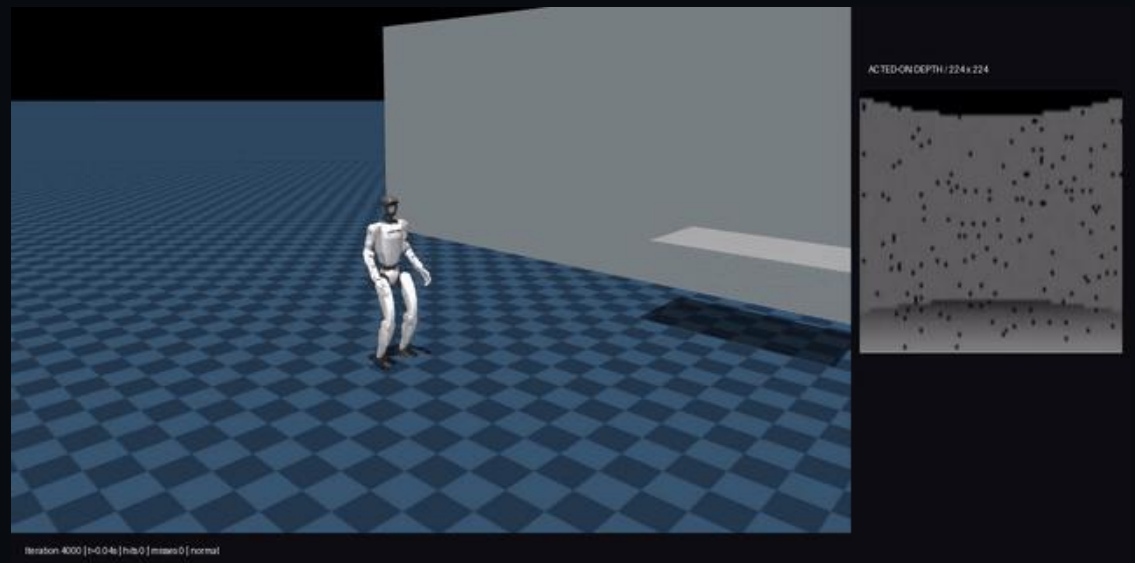
Frozen Theia-Tiny · 16 frames at 25 Hz · learned 128-D percept

Training Depth Dodge

512
parallel worlds

4,000
PPO iterations

49.2M
transitions



Simulation rollout after 4,000 PPO iterations

Learned

Temporal adapter
+ 7 rank-16 LoRAs

Auxiliary supervision

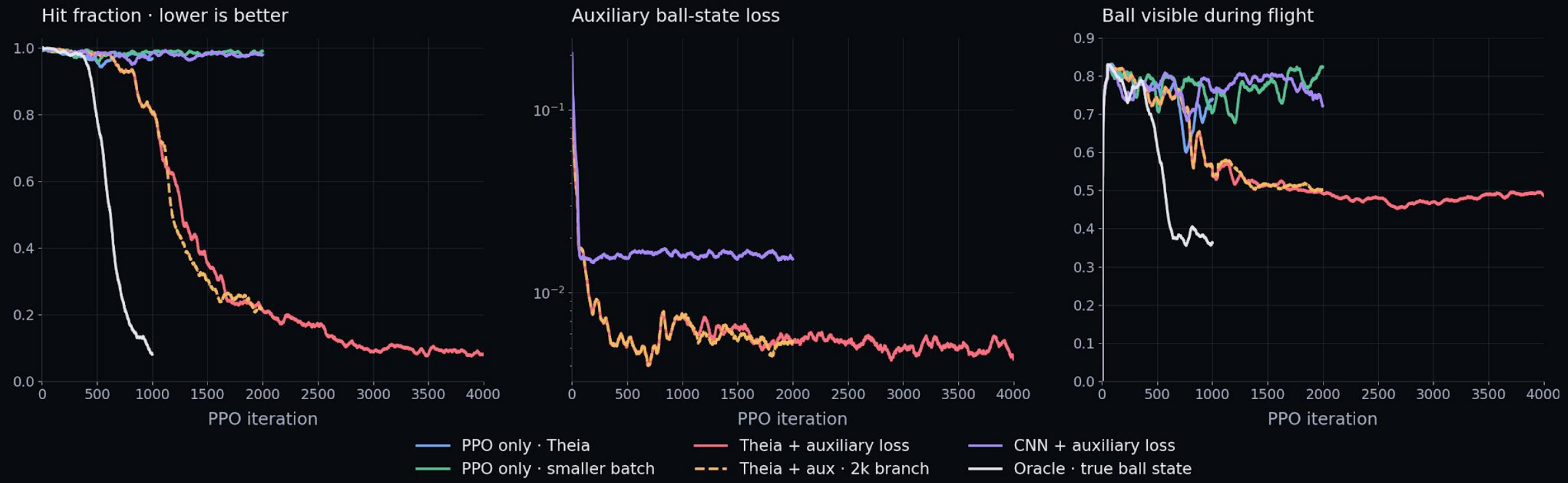
Ball position and velocity
Auxiliary loss weight 0.05

Frozen Theia and SONIC base · randomized dynamics and depth

1 × H100 80 GB · 14.05 h logged iteration time

What makes visual learning work?

PPO alone is not enough in this experiment



Theia + ball-state supervision learns the dodge

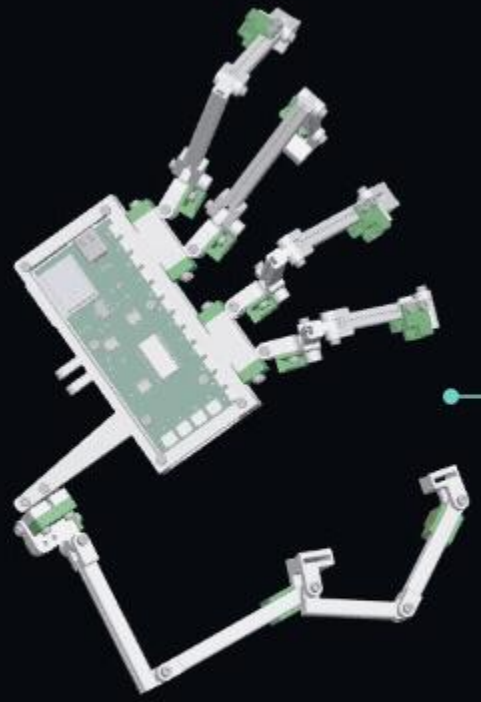
Training curves · 50-iteration smoothing · camera visibility is a diagnostic

Depth Dodge evaluation and sim-to-real

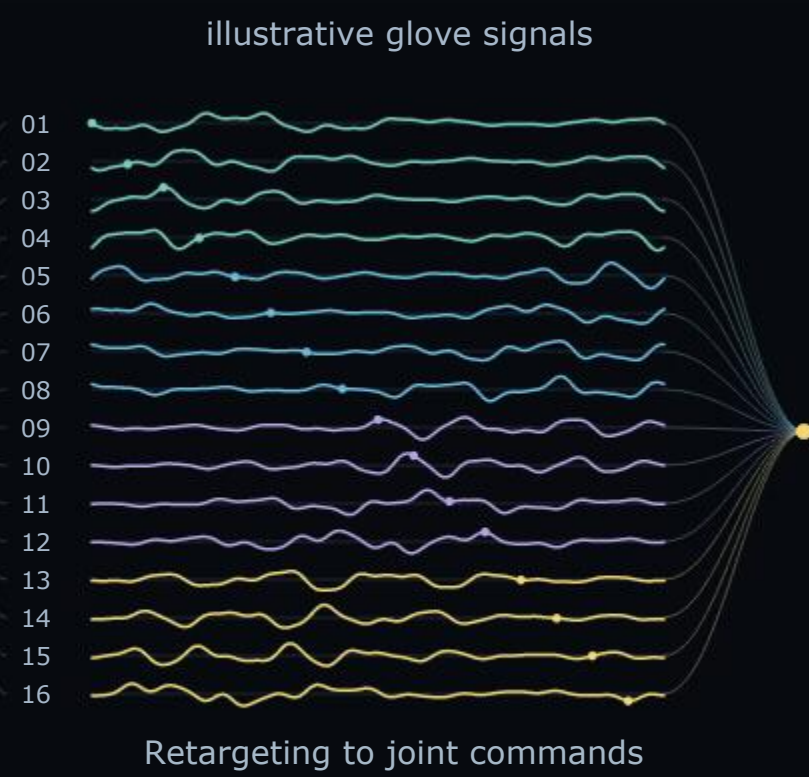




Open-source humanoid hardware



Homunculus glove





HopeJR hand

We also build and release hardware for others to use and improve.

Humanoid datasets on the Hub

HIW-500



Teleoperation in real homes
Head/wrist cameras
States + action traces

HLM-12



12 whole-body tasks
OpenHLM demo preview
Vision + state/action data

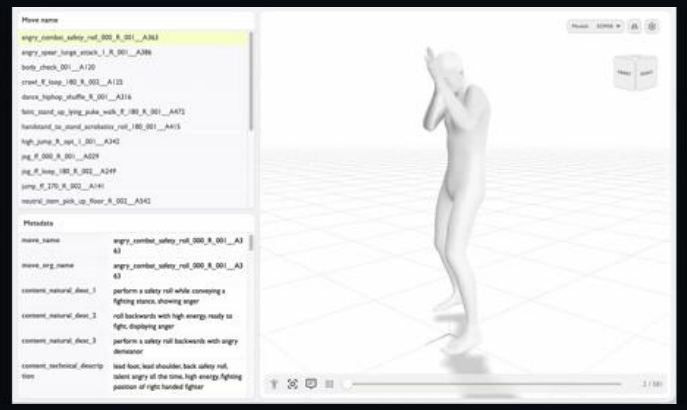
UniFoLM-WBT



Household loco-manipulation
Stereo + wrist cameras
Body/hand + base commands

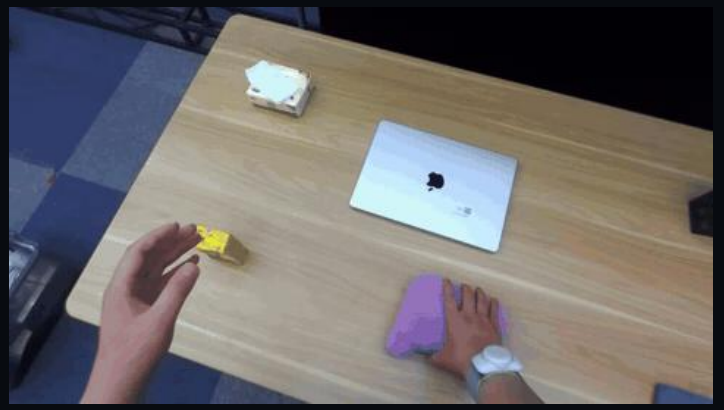
Humanoid datasets on the Hub

BONES-SEED



Human motion capture
Retargeting for G1 control

EgoHumanoid



Human + robot demonstrations
Aligned for VLA co-training

tracker-pov



First-person video
7-point IMU tracking
Motion and visual context

Next steps

ASIMOV

UniFoLM

Broader controllers and datasets

Force-feedback devices



Asimov 1 · Menlo Research

Open interfaces, data and hardware that others can build on